TP05 : Manipulation des Capteurs sous Android

Matière : ATELIER DEVELOPPEMENT MOBILE AVANCE	**Classes : SEM31**

Ce TP est rédigé avec des questions/réponses Gemini.

Objectif du TP

Ce premier TP a pour objectif de permettre à l'étudiant de se familiariser avec l'API matérielle d'Android (**SensorManager**). À travers les différents exercices, l'étudiant apprendra à :

1. **Découvrir et inventorier** les capteurs disponibles sur un terminal Android physique ou émulé.
2. **Capturer en temps réel** les flux de données continus émis par un capteur (en l'occurrence, l'accéléromètre) et mettre à jour l'interface utilisateur de manière dynamique.
3. **Persister les données localement** sur l'appareil en utilisant une base de données **SQLite**.
4. **Communiquer avec un serveur distant** via des requêtes HTTP (en utilisant la bibliothèque **Volley**) afin de synchroniser ou de sauvegarder les mesures dans une base de données distante.

Exercice1 : Lister les capteurs de l'appareil

L'objectif ici est d'obtenir le gestionnaire de capteurs (**SensorManager**) et de lister l'ensemble des capteurs intégrés au terminal, puis d'afficher leurs informations principales dans la console de log (**Log.d**).

1- Lancer Android Studio et créer un nouveau projet « ListerCapteurs ».

2- Télécharger une icone « sensor.png » à partir de « https://www.magnific.com/icon/ »

3- Modifier l'icône de lancement de l'application à « sensor.png » (res/drawable et AndroidManifest.xml).

4- Exécuter l'application.

5- Taper le code suivant :

```
public class MainActivity extends AppCompatActivity {

    private static final String TAG = "TP_Capteurs";
    private SensorManager sensorManager;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // 1. Initialisation du SensorManager
        sensorManager = (SensorManager) getSystemService(SENSOR_SERVICE);

        // 2. Récupération de la liste de tous les capteurs
        List<Sensor> sensorList = sensorManager.getSensorList(Sensor.TYPE_ALL);
```

```
        // 3. Affichage dans le Log.d avec interpolation et retour à la ligne
        Log.d(TAG, "=== Liste des capteurs disponibles (" +
                   "${sensorList.size}) ===")

        for (sensor in sensorList) {
            Log.d(TAG, "Nom : ${sensor.name} " +
                       "| Type : ${sensor.stringType} " +
                       "| Vendeur : ${sensor.vendor}")
        }
    }
}
```

6- Exécuter l'application.

7- Afficher d'autre champs de « sensor » et exécuter.

8- Changer Sensor.TYPE_ALL par Sensoe.TYPE_ACCELEROMETER ; exécuter et Valider le travail.

Exercice 2 : Affichage de l'accélération (Log et TextView)

Cet exercice consiste à écouter les variations de l'accéléromètre en temps réel. L'activité doit implémenter l'interface **SensorEventListener**, s'enregistrer dans **onResume()** et se désenregistrer dans **onPause()** pour économiser la batterie. L'exercice utilise les classes **SensorManager**, **Sensor** et **SensorEvent**.

1- Lancer Android Studio et créer un nouveau projet « AffichageAcceleration ».

2- Importer les layouts de l'application (copier les et les coller dans res/layout).

3- Ajouter le viewBinding à build.gradle.kts puis Sync now.

4- Ajouter le code de MainActivity.

```
class MainActivity : AppCompatActivity(), SensorEventListener {

    companion object {
        private const val TAG = "TP_Accelero"
    }

    private lateinit var binding: ActivityMainBinding
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }
```

```
private fun initApp() {
    // 1. Initialisation du View Binding et affectation de la vue
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    // 2. Initialisation des TextViews avec une valeur par défaut
    binding.txtX.text = "X : 0.0"
    binding.txtY.text = "Y : 0.0"
    binding.txtZ.text = "Z : 0.0"

    // 3. Initialisation du SensorManager et du capteur d'accélération
    sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
}
```

```
override fun onResume() {
    super.onResume()
    // Exécute le bloc uniquement si l'accéléromètre n'est pas null
    // (évite un NullPointerException si le terminal n'intègre pas ce capteur)
    accelerometer?.let {
        sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_NORMAL)
    }
}

override fun onPause() {
    super.onPause()
    // Désenregistrement dans onPause() pour couper le capteur
    // en arrière-plan et économiser la batterie
    sensorManager.unregisterListener(this)
}
```

```
override fun onSensorChanged(event: SensorEvent?) {
    // Traitement des variations de l'accéléromètre en temps réel
    if (event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {
        val x = event.values[0]
        val y = event.values[1]
        val z = event.values[2]

        // Mise à jour des TextViews avec les valeurs actuelles du capteur
        binding.txtX.text = "X : %.2f".format(x)
        binding.txtY.text = "Y : %.2f".format(y)
        binding.txtZ.text = "Z : %.2f".format(z)
    }
}
```

```
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
        // Méthode obligatoire de l'interface SensorEventListener,
        // laissée vide ici
    }
}
```

Exercice3 : Suivi en temps réel de l'accéléromètre et journalisation périodique sous SQLite

Cet exercice consiste à écouter les variations de l'accéléromètre en temps réel et de les enregistrer dans une base de données SQLite qui contient une table Mesure.

1- Lancer Android Studio et créer un nouveau projet « JournalLocalAcc ».
2- Importer les layouts de l'application (copier les et les coller dans res/layout).
3- Ajouter le viewBinding à build.gradle.kts puis Sync now.
4- Importer les classes MainActivity et SQLiteMesure.
5- Compléter les deux méthodes de SQLiteMesure (insererMesure et listerMesure).
6- Dans MainActivity déclarer et initialiser un objet de type SQLiteMesure.
7- Ajouter les modifications nécessaires à onSensorEventChanged pour enregistrer.

```
// Vérification si le Switch d'enregistrement est activé
val isEnregistrementActif = binding.switchEnregistrement.isChecked

if (isEnregistrementActif) {
    val tempsActuel = System.currentTimeMillis()
    // Vérifie si 2 secondes se sont écoulées depuis le dernier enregistrement
    if (tempsActuel - dernierTempsEnregistrement >= INTERVAL_ENREGISTREMENT_MS) {
        dernierTempsEnregistrement = tempsActuel

        // Récupération de la date et heure courante formatée
        val dateFormat = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
        val dateTimeStr = dateFormat.format(Date())
```

```
        // Insertion dans la base de données SQLite via SQLiteMesure
        sqliteMesure.insererMesure(
            typeCapteur = "ACCELEROMETER",
            dateTime = dateTimeStr,
            x = x,
            y = y,
            z = z
        )
    }
```

8- Ajouter les modifications nécessaires à ajouterEcouteur pour lister les mesures.

```
private fun ajouterEcouteur() {
    // 5. Gestion du clic sur le bouton pour lister les mesures dans le Logcat
    binding.btnListerMesures.setOnClickListener {
        sqliteMesure.listerMesures()
    }
}
```

Exercice4 : Suivi en temps réel de l'accéléromètre et journalisation périodique sous une Base de données distante

Cet exercice consiste à écouter les variations de l'accéléromètre en temps réel et de les enregistrer dans une base de données distante en passant par une application web.

1- Lancer Android Studio et créer un nouveau projet « JournalDistantAcc ».
2- Importer les layouts de l'application (copier les et les coller dans res/layout).
3- Ajouter le viewBinding à build.gradle.kts puis Sync now.
4- Compléter le code de MainActivity pour afficher les variations de l'accélération.
5- Ajouter au Gradle l'implémentation de la bibliothèque volley puis Sync now.
6- Ajouter la permission INTERNET.
7- Ajouter usesCleartextTraffic=true.
8- Importer les classes MainActivity et MesureNetworkHelper.
9- Compléter les deux méthodes de MesureNetworkHelper :
    a. envoyerMesureDistant
    b. listerMesuresDistant.
10- Dans MainActivity déclarer et initialiser un objet de type MesureNetworkHelper.
11- Ajouter les modifications nécessaires à onSensorEventChanged pour enregistrer.
12- Ajouter les modifications nécessaires à ajouterEcouteur pour lister les mesures.